Final Exam : Remove Duplicates : KEY

CSE 130: Algorithm Design

Pseudocode

```
PROMPT for filename
GET filename

OPEN file
names ← file.read()                          A

i_write ← 0                                  B

FOR i_read ← 0 … names.size()                C
   ASSERT 0 ≤ i_read < names.size()
   IF names[i_read] ≠ names[i_read + 1]       D
      ASSERT i_write ≤ i_read
      ASSERT 0 ≤ i_write < names.size()
      names[i_write] ← names[i_read]
      i_write += 1                           E

FOR i_pop ← i_write-1 … names.size()
   names.pop()                               F

FOR name in names
   PUT name                                  G
```

Quality

Algorithmic efficiency:

A. O(n) Because each name needs to be read from the file
B. O(1) Initialization takes a fixed amount of time
C. O(n) Because each name is visited once
D. O(n) The IF statement is O(1) but it is performed O(n) times
E. O(n) Either done once if the entire list is duplicates or O(n) if there are no duplicates
F. O(n) Done zero times if there are no duplicates, done O(n) if there are all duplicates. From the problem definition, this could be as much as 2/3s of the time if everyone is registered, an invited speaker, and a member.
G. O(n) once for every non-duplicate. Could be 1 if everyone is a duplicate.

Understandability

Obvious if comments are added, straightforward as it currently stands in pseudocode

- file: clearly a file variable
- names: clearly a collection of names.
- i_write: clearly an index into what we are writing
- i_read: clearly an index of what we are reading form
- i_pop: clearly an index into how many pops are performed.

Malleability

The filename is prompted from the user, the data is provided from a file. This is configurable malleability.

Quality

Asserts

Asserts verify the indices are within range. Also, assert that i_read ≥ i_write.

Trace

	names	i_read	i_write	i_pop	Output
A	["A", "B", "B", "C"]	/	/	/	
B	["A", "B", "B", "C"]	/	0	/	
C	["A", "B", "B", "C"]	0	0	/	
D	["A", "B", "B", "C"]	0	0	/	
E	["A", "B", "B", "C"]	0	1	/	
C	["A", "B", "B", "C"]	1	1	/	
D	["A", "B", "B", "C"]	1	1	/	
C	["A", "B", "B", "C"]	2	1	/	
D	["A", "B", "B", "C"]	2	1	/	
E	["A", "B", "B", "C"]	2	2	/	
C	["A", "B", "B", "C"]	3	2	/	
D	["A", "B", "B", "C"]	3	2	/	
E	["A", "B", "C", "C"]	3	3	/	
F	["A", "B", "C"]	/	3	2	
G	["A", "B", "C"]	/	/	/	A
G	["A", "B", "C"]	/	/	/	B
G	["A", "B", "C"]	/	/	/	C